# National University of Computer and Emerging Sciences



## ASSIGNMENT REPORT
## End-to-End ML Versioning Pipeline with Git, DVC & Google Drive

**Course:** MLOPS
**Assignment:** Assignment 3 – Fashion-MNIST ANN Pipeline Versioning
**Repository:** github.com/Anasnaveed69/fashion-ann-pipeline
**Target:** ≥ 85% Test Accuracy


| Name | Roll Number | Sub-section |
|---|---|---|
| Anas Naveed Butt | 23L-0764 | 7-D |

Department of Computer Science
FAST-NU, Lahore, Pakistan

# 1. Executive Overview and Problem Statement

For this assignment I built a reproducible ML versioning and orchestration pipeline for classifying the Fashion-MNIST dataset, which contains 70,000 grayscale images of size 28×28 spread across 10 clothing categories. My goal was to train an Artificial Neural Network (ANN) that reaches at least 85% accuracy on the test set, while keeping the entire workflow reproducible.

Along with the model itself, the assignment required me to practise advanced Git history operations, set up a multi-stage DVC pipeline backed by Google Drive remote storage, and resolve a situation where a code conflict and a DVC data-pointer conflict occur at the same time. All work is pushed to my GitHub repository **fashion-ann-pipeline** (username: Anasnaveed69).

# 2. Part A – Git Fundamentals and Advanced Commands

## A3. Git Log Variants

I tried several variants of `git log` to see what extra information each one gives compared to the plain command:

| Command | What It Showed Me |
|---|---|
| git log --oneline --graph --all | A compact ASCII graph of every branch with short 7-character hashes. This made it easy to see exactly where branches split, merged and were rebased. |
| git log --stat -3 | For the last three commits, the files changed and the number of lines added and removed, without printing the full patch. |
| git log -p -1 | The complete line-by-line diff introduced by the most recent commit (HEAD). |
| git log main..dev | Only the commits that exist on dev but have not yet been merged into main. |

## A4. Git Diff Variants and Two-Dot vs Three-Dot Comparison

- **git diff:** compares my working tree with the staging area, so it shows edits I have not staged yet.
- **git diff --staged:** compares what is staged with the last commit (HEAD), i.e. what will go into the next commit.
- **git diff main..dev (two-dot):** directly compares the current tip of main with the current tip of dev.
- **git diff main...dev (three-dot):** first finds the common ancestor (merge-base) of main and dev, then compares it with dev. This isolates only the changes that dev introduced after the two branches diverged, which is more useful when reviewing a feature branch.

## A5. Git Stash Workflow

While I was in the middle of editing `preprocess.py` on dev, I needed to switch to main urgently. Instead of committing half-finished code, I used the stash:

1. `git stash push -m "WIP: mid-edit on preprocess.py"` – saved my changes and left the working tree clean.
2. `git checkout main` – switched to main, checked its state, then returned with `git checkout dev`.
3. `git stash list` – confirmed the entry `stash@{0}: On dev: WIP: mid-edit on preprocess.py`.
4. `git stash pop` – restored my changes and continued editing exactly where I had left off.

## A7. Git Reset: Soft vs Hard

To compare the two reset modes safely, I created a throwaway branch `scratch-experiments` and made two test commits on it:

- **git reset --soft HEAD~1:** HEAD moved back one commit, but my working directory and staging area were untouched. `git status` still showed `scratch1.txt` as a staged new file, ready to be committed again.
- **git reset --hard HEAD~1:** HEAD moved back one commit and the changes were also removed from both the index and the working directory, so `scratch2.txt` was deleted from disk.

## A6 & A8. Rebase and Repository Reorganization

**A6 (Rebase):** I created a `hotfix` branch from main, added an environment reproduction note to the README, and merged it back into main. I then switched to dev and ran `git rebase main`, which re-applied dev's six incremental commits on top of the latest commit on main, keeping the history linear and clean.

**A8 (Reorganization):** I used `git mv` to move the loose pipeline scripts (prepare, preprocess, train, evaluate) into a `src/` folder, and removed the scratch notes with `git rm scratch_notes.txt`, so both operations were recorded properly in the history.

### Complete Repository Graph

The final history of my repository, obtained with `git log --oneline --graph --all`, is shown below:

```
* 6bc7ff6 chore(dvc): Lock finalized reconciled pipeline state
*   ccf6b5a merge: Resolve normalization conflict and sync DVC data
|\           pointer (main & teammate-sim)
| * 8aa9431 feat(teammate-sim): Implement z-score standardization &
|           update processed data hash
* | 1480b41 feat(main): Implement centered [-1, 1] normalization &
|/          update processed data hash
* 4d1ac53 chore(dvc): Configure default Google Drive remote storage
*   11aff19 merge: Merge dev feature branch into main
|\
| * 6205680 feat(params): Tune dense_units to 256 and update v2 model
|           metrics (tag: v2)
| * 6a19b25 feat(dvc): Configure DVC pipeline and lock initial v1 model
|           artifacts (tag: v1)
| * 1f75ed7 refactor: Reorganize pipeline scripts into src and remove
|           scratch notes
| * 089a633 feat: Define central hyperparameters configuration in params.yaml
| * af3192f feat: Add model evaluation and confusion matrix generation script
| * dedc67e feat: Add ANN model training script with Adam optimizer
| * 4a85be5 feat: Add data preprocessing and validation split script
| * 7fde393 feat: Add data preparation script and scratch notes
|/
* e92d3f3 fix(docs): Add environment reproduction note to README (hotfix merged)
* 45ff5e7 Initial commit: Project setup with README.md, .gitignore, and
          requirements.txt
```

## 3. Part B – Modular TensorFlow ANN Pipeline

I split the pipeline into four independent scripts inside `src/`. Each one can be run from the command line on its own, and none of them contains hard-coded hyperparameters; all values are read from `params.yaml`.

| Script | What It Does | Inputs / Outputs |
|---|---|---|
| src/prepare.py | Downloads Fashion-MNIST through keras.datasets and saves the raw images and labels as NumPy arrays. | Out: data/raw/{train,test}_{images,labels}.npy |
| src/preprocess.py | Normalizes the pixel values and splits the training data into stratified train and validation sets using params.yaml. | In: data/raw<br>Out: data/processed/{X,y}_{train,val,test}.npy |
| src/train.py | Builds a Sequential ANN (Flatten → Dense(ReLU) → Dropout → Dense(10, Softmax)) and trains it with Adam. | In: data/processed, params.yaml<br>Out: models/model.h5, models/history.csv |

| Script | What It Does | Inputs / Outputs |
|---|---|---|
| src/evaluate.py | Evaluates test loss and accuracy, plots a confusion matrix heatmap and writes the metrics to a JSON file. | In: models/model.h5, data/processed<br>Out: metrics.json, models/confusion_matrix.png |

## Model Architecture

- **Input layer:** Flatten (28×28 → 784 features)
- **Hidden layer:** Dense, 256 units, ReLU activation
- **Regularization:** Dropout with rate 0.20
- **Output layer:** Dense, 10 units, Softmax activation
- **Optimizer and loss:** Adam (learning rate 0.001) with Sparse Categorical Crossentropy

## Results on the Test Set

The final (v2) model reached a **test accuracy of 88.17%** with a **test loss of 0.3435**, which is above the 85% target. The confusion matrix in Figure 1 shows a strong diagonal across all ten classes. Most of the remaining errors are between visually similar upper-body items such as Shirt, T-shirt/top, Pullover and Coat, which is expected for a simple fully connected network that does not use spatial features.

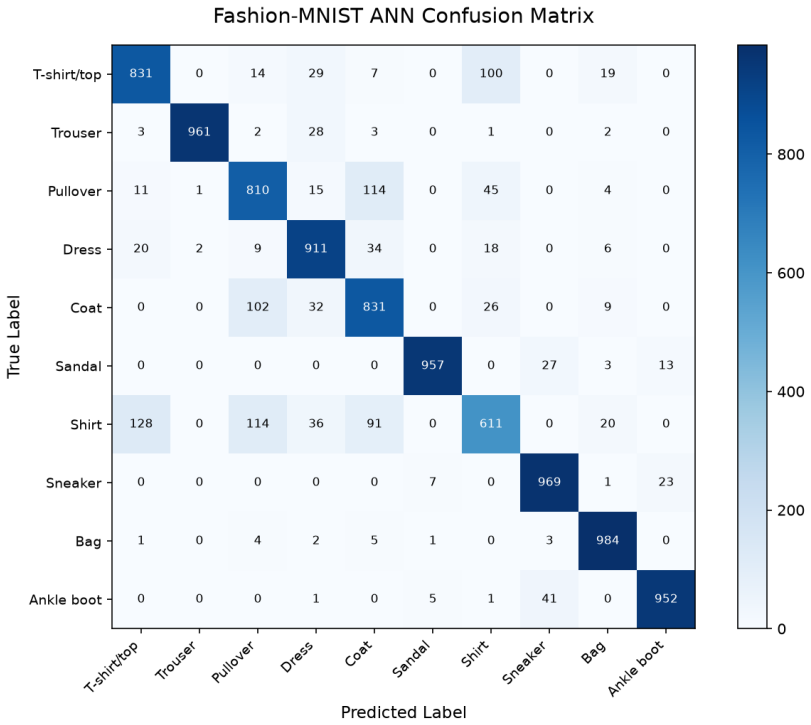



*Figure 1: Confusion matrix of the v2 model on the Fashion-MNIST test set*

# 4. Part C & D – DVC Pipeline Configuration and Reproducibility

I initialized DVC on the dev branch and added Google Drive as the default remote storage:

```
dvc remote add -d gdriveremote gdrive://1ke8PvXC583PccUvoobOeMrqiywJFURZP
```

Credentials are kept only in local configuration and are excluded through `.gitignore`, so nothing sensitive is committed to GitHub.

### Pipeline Specification (params.yaml and dvc.yaml)

```
# params.yaml
preprocess:
  test_size: 0.2
  seed: 42
  normalization: standard
train:
  dense_units: 256
  dropout_rate: 0.2
  learning_rate: 0.001
  epochs: 10
  batch_size: 64
```

```
# dvc.yaml
stages:
  prepare:
    cmd: python src/prepare.py
    deps: [src/prepare.py]
    outs: [data/raw]
  preprocess:
    cmd: python src/preprocess.py
    deps: [data/raw, src/preprocess.py]
    params: [preprocess.seed, preprocess.test_size]
    outs: [data/processed]
  train:
    cmd: python src/train.py
    deps: [data/processed, src/train.py]
    params: [train.dense_units, train.epochs, ...]
    outs: [models/history.csv, models/model.h5]
  evaluate:
    cmd: python src/evaluate.py
    deps: [data/processed, models/model.h5, src/evaluate.py]
    outs: [models/confusion_matrix.png]
    metrics:
      - metrics.json: {cache: false}
```

## D3 – Initial End-to-End Run

On the first `dvc repro`, all four stages executed in order because nothing had been cached yet:

```
$ dvc repro
Running stage 'prepare':
> python src/prepare.py
[PREPARE] Downloaded Fashion-MNIST to data/raw
Updating lock file 'dvc.lock'
Running stage 'preprocess':
> python src/preprocess.py
[PREPROCESS] Normalized & split to data/processed
Updating lock file 'dvc.lock'
Running stage 'train':
> python src/train.py
[TRAIN] Built ANN (dense_units=128)
[TRAIN] Epoch 10/10 - val_loss: 0.3234
Updating lock file 'dvc.lock'
Running stage 'evaluate':
> python src/evaluate.py
[EVALUATE] Test Loss: 0.3641 | Test Accuracy: 87.02%
Updating lock file 'dvc.lock'
```

## D4 – Run After Changing a Hyperparameter

I then changed `train.dense_units` from 128 to 256 in `params.yaml` and ran `dvc repro` again:

```
$ dvc repro
Stage 'prepare' didn't change, skipping
Stage 'preprocess' didn't change, skipping
Running stage 'train':
> python src/train.py
[TRAIN] Loading data/processed
[TRAIN] Built ANN (dense_units=256)
[TRAIN] Epoch 10/10 - val_loss: 0.3038
Updating lock file 'dvc.lock'
Running stage 'evaluate':
> python src/evaluate.py
[EVALUATE] Test Loss: 0.3435 | Test Accuracy: 88.17%
Updating lock file 'dvc.lock'
Use 'dvc push' to send updates to remote storage.
```

## Why Some Stages Were Skipped and Others Re-ran

In the D4 run, **prepare** and **preprocess** were skipped, while **train** and **evaluate** were executed again. DVC treats the pipeline as a dependency graph and, for each stage, computes MD5 hashes of its code, its data dependencies and the specific parameters it uses. These hashes are compared with the values stored in `dvc.lock`.

Since I only changed `train.dense_units`, the hashes for prepare and preprocess were exactly the same as before, so DVC skipped them and saved compute time. The train stage depends on that parameter, so it re-ran and produced new weights in `models/model.h5`. Because `model.h5` is a dependency of evaluate, its hash changed as well, and DVC automatically ran evaluate afterwards.

### Comparison of v1 and v2

| Version | Dense Units | Epochs | Dropout | Test Loss | Test Accuracy | Git Tag |
|---|---|---|---|---|---|---|
| v1 (Baseline) | 128 | 10 | 0.20 | 0.3641 | 87.02% | v1 |
| v2 (Tuned) | 256 | 10 | 0.20 | 0.3435 | 88.17% (+1.15%) | v2 |

Doubling the hidden layer size improved accuracy by 1.15 percentage points and reduced the test loss, without any change to the data or preprocessing stages.

## 5. Part E – Simulated Collaboration and Conflict Resolution

To simulate working in a team, I created a branch `teammate-sim` and made it implement a different normalization technique from main. Both branches regenerated the processed dataset, so they ended up with different code in `src/preprocess.py` and different data hashes in `dvc.lock`.

### E3. Simultaneous Code and Data-Pointer Conflict

When I merged `teammate-sim` into main, Git reported conflicts in both files:

```
$ git merge teammate-sim
Auto-merging dvc.lock
CONFLICT (content): Merge conflict in dvc.lock
Auto-merging src/preprocess.py
CONFLICT (content): Merge conflict in src/preprocess.py
Automatic merge failed; fix conflicts and then commit the result.
```

The conflict markers in the code file showed main's centered [-1, 1] scaling against the teammate's z-score standardization:

**Conflict in src/preprocess.py**

```
<<<<<<< HEAD (branch 'main')
print('[PREPROCESS] Centered Min-Max scaling [-1, 1]')
x_train_norm = (train_images / 127.5) - 1.0
x_test_norm = (test_images / 127.5) - 1.0
=======
print('[PREPROCESS] Z-score standardization (mean=0, std=1)')
mean_val = np.mean(train_images)
std_val = np.std(train_images) + 1e-7
x_train_norm = (train_images - mean_val) / std_val
x_test_norm = (test_images - mean_val) / std_val
>>>>>>> teammate-sim
```

In `dvc.lock`, the conflict was on the MD5 hash of the `data/processed` output, since each branch had produced a different version of the dataset:

**Conflict in dvc.lock (data/processed pointer)**

```
outs:
- path: data/processed
  hash: md5
<<<<<<< HEAD (branch 'main')
  md5: 14a913ef7eb8c8be13a464958fc2263e.dir
=======
  md5: 49ecdde221153d9aa60942819391cdd2.dir
>>>>>>> teammate-sim
  size: 219590768
  nfiles: 6
```

### E4 & E5. Resolution Steps and Verification

I resolved the conflict in the following steps:

1. **Code reconciliation:** Instead of discarding one approach, I rewrote `src/preprocess.py` with a single normalization function that supports both methods, with the choice controlled by the `normalization` key in `params.yaml`.
2. **Data pointer resolution:** I removed the conflict markers in `dvc.lock` and kept main's verified data hash (`14a913ef...`).
3. **Workspace sync:** I ran `dvc checkout` so the files in `data/processed/` matched the resolved hash.
4. **Merge commit:** I staged the resolved files with `git add src/preprocess.py dvc.lock` and completed the merge (commit `ccf6b5a`).
5. **Reproducibility check:** I ran `dvc repro` from the merged state. The preprocess stage regenerated the data cleanly with the reconciled code, and I committed the updated lock file as `6bc7ff6`. Running DVC again afterwards confirmed that the data and pipelines were up to date.

## 6. Deliverables and Submission Checklist

| Deliverable | Status | Details |
|---|---|---|
| GitHub Repository | Completed | Full, unsquashed history of 16 commits across main, dev, hotfix and teammate-sim.<br>Remote: https://github.com/Anasnaveed69/fashion-ann-pipeline.git |
| Google Drive Remote | Configured | gdriveremote registered in .dvc/config (gdrive://1ke8PvXC583PccUvoobOeMrqiywJFURZP). All artifacts uploaded with dvc push. |

| Deliverable | Status | Details |
|---|---|---|
| PDF Report | Completed | This report, including configuration files, terminal logs, the repository graph, results tables and the confusion matrix. |